



Problem Solving With C - UE25CS151B

C Standards and Guidelines

Prof. Sindhu R Pai

PSWC Theory Anchor, Feb-May, 2025

Department of Computer Science and Engineering

PROBLEM SOLVING WITH C

C Standards and Guidelines



Why Follow Standards and Guidelines to code in C?

- Ensures readability and maintainability of code.
- Reduces debugging and development time, especially when multiple developers are working on the same code.
- Promotes consistency in code formatting.
- In certain cases, even improves the runtime and security of your code.

PROBLEM SOLVING WITH C

C Standards and Guidelines



Standards and Best Practices

- Consistent Formatting
- Naming Conventions
- Commenting and Documentation
- Header File Practices
- Avoidance of Undefined Behavior
- Error Handling
- Memory Management
- Maintain Modularity
- Security Practices
- Portability & Standards Compliance
- Static Analysis & Testing
- Established Guidelines/Standards

PROBLEM SOLVING WITH C

C Standards and Guidelines



Code Formatting Guidelines

- **Use indentation:** 2-4 spaces per level, no tabs¹. Tabs are avoided as a standard practice since different editors and IDEs can display/process tabs differently.
- Follow a uniform brace style
- Keep lines to a reasonable length for readability - Avoid long lines (greater than 80 characters)¹.

Naming Conventions

- Use meaningful and descriptive names for variables and function names.
- Use consistent naming patterns (e.g., snake_case for functions/variables, UPPER_CASE for macros)

¹ - Source: <https://google.github.io/styleguide/cppguide.html>

PROBLEM SOLVING WITH C

C Standards and Guidelines



Documentation

- Add clear, concise comments for complex logic
- Include file headers with purpose, author, date, and version.
- Maintain external documentation where needed.

Header file Practices

- Use include guards (`#ifndef`, `#define`, `#endif`) to avoid multiple inclusions
- Separate declarations (in headers) from implementations (in source files)

PROBLEM SOLVING WITH C

C Standards and Guidelines



Avoidance of Undefined Behaviour

- Write code that avoids undefined behaviors (e.g., out-of-bounds access, null pointer dereferences)
- Validate assumptions and inputs rigorously

Error Handling

- **Check return values of functions** (malloc, scanf, etc.) to account for any unforeseen behaviour.
- Use **errno** and **perror()** for error messages that may pop up in your code⁵.
- Use **assertions (assert.h)** for debugging⁶.
- **Avoid using the 'goto' construct** for any flow control⁶.

5 - Source: <https://www.gnu.org/prep/standards/>

6 - Source: <https://www.oreilly.com/library/view/code-complete-second/0735619670/>

PROBLEM SOLVING WITH C

C Standards and Guidelines



Efficient Memory Management

- Always **initialise pointers**².
- **Do not dereference NULL pointers.**
- Ensure every memory allocation (malloc(), calloc()) is paired with a corresponding free() => Avoid any memory leaks by **always freeing dynamically allocated memory**².
- Employ cleanup techniques to avoid resource mismanagement
- Use static/global variables only if absolutely necessary. **Use local variables wherever possible.**

² - Sources: <https://docs.microsoft.com/en-us/security/>; <https://wiki.sei.cmu.edu/confluence/display/c/SEI+CERT+C+Coding+Standard>

PROBLEM SOLVING WITH C

C Standards and Guidelines



Maintain Modularity

- Decompose code into small, reusable functions
- Limit the scope of variables (using static for file-local scope) to reduce complexity
- **Use functions** to avoid redundant segments of code throughout the program.
- Keep all functions **short, modular**, and primarily focused on a **single purpose**³.
- Use **short and informative comments** without being redundant.
- Use **'const'** and **'#define'** for constant values being used⁴.

3 - Source: <https://www.oreilly.com/library/view/clean-code/9780136083238/>

4 - Source: <https://www.misra.org.uk/misra-c/>

PROBLEM SOLVING WITH C

C Standards and Guidelines



Security Practices

- Validate all inputs to prevent buffer overflows and other vulnerabilities.
- Prefer safe functions (e.g., `strncpy()` over `strcpy()`) when handling strings and buffers.

Portability and Standards Compliance

- Adhere to a specific C standard (e.g., C99, C11) to maximize portability.
- Avoid platform-specific behavior; use conditional compilation when needed.

PROBLEM SOLVING WITH C

C Standards and Guidelines



Static Analysis and Testing

- Enable compiler warnings to catch potential issues early
- Use static analysis tools to identify bugs and security vulnerabilities
- Write unit tests for critical functions to ensure reliability

Following Established Standards

- Consider industry standards like MISRA C or the CERT C Secure Coding Standard for safety-critical projects
- Adhere to project-specific coding guidelines for consistency and quality



THANK YOU

Department of Computer Science and Engineering

Prof. Sindhu R Pai - sindhurpai@pes.edu

Ack: Teaching Assistant – Advait Sanil Kumar